Karlova Univerzita
Přírodovědecká fakulta
Katedra aplikované geoinformatiky a kartografie

Algoritmy počítačové kartografie

Geometrické vyhledávání bodu

Aliaksandra Sparysh
Jolana Václavková

Praha 2026

Úloha č. 1: Geometrické vyhledávání bodu

Vstup: Souvislá polygonová mapa n polygonů $\{P_1, \ldots, P_n\}$, analyzovaný bod q.

Výstup: P_i, $q \in P_i$.

Nad polygonovou mapou implementujete Ray Crossing Algorithm pro geometrické vyhledání incidujícího polygonu obsahujícího zadaný bod q.

Nalezený polygon graficky zvýrazněte vhodným způsobem (např. vyplněním, šrafováním, blikáním). Grafické rozhraní vytvořte s využitím frameworku QT.

Pro generování nekonvexních polygonů můžete navrhnout vlastní algoritmus či použít existující geografická data (např. mapa evropských států).

Polygony budou načítány z textového souboru ve Vámi zvoleném formátu. Pro datovou reprezentaci jednotlivých polygonů použijte špagetový model.

Námi zpracované úlohy:

Krok	**Hodnocení**
Detekce polohy bodu rozlišující stavy uvnitř, vně polygonu.	10b
Analýza polohy bodu (uvnitř/vně) metodou *Winding Number Algorithm*.	+10b
Ošetření singulárního případu u *Winding Number Algorithm*: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u *Ray Crossing Algorithm*: bod leží na hraně polygonu.	+5b
Ošetření singulárního případu u obou algoritmů: bod je totožný s vrcholem jednoho či více polygonů.	+2b
Zvýraznění všech polygonů pro oba výše uvedené singulární případy.	+3b
Rychlé vyhledání potenciálních polygonů (bod uvnitř min-max boxu).	+10b
Řešení pro polygony s dírami.	+10b
Načtení vstupních dat ze `*.shp`.	+10b
Zpracování textové zprávy v LaTeXu	+5b
Max celkem:	**70b**

Čas zpracování: 2 týdny.

Popis problému

Point location problem

Často řešenou úlohou v oblasti počítačové grafiky, digitální kartografie, či geografických informačních systémů (GIS) je určování vzájemného topologického vztahu mezi zadaným referenčním bodem a uzavřenou oblastí. Tento úkol, v odborné literatuře standardně označovaný jako Point-in-Polygon Problem (PIPP), poskytuje exaktní informaci o tom, zda se zkoumaný bod nachází v interiéru (uvnitř), exteriéru (vně), případně přímo na hranici souvislého geometrického útvaru (polygonu).

Náročnost algoritmického aparátu potřebného pro řešení této úlohy se diametrálně liší v závislosti na geometrických vlastnostech analyzovaného útvaru. Zatímco zjištění polohy bodu vůči striktně konvexním polygonům lze provést pomocí jednodušších a výpočetně méně náročných metod (např. testováním polohy bodu vůči polorovinám), v praxi se převážně pracuje s polygony nekonvexními. Tyto složitější útvary vyžadují robustnější analytické přístupy, které dokáží zohlednit specifické rozložení vrcholů a případnou existenci děr.

Předložená práce se zabývá řešením problému PIPP nad komplexní polygonovou mapou. Geografická data jsou v tomto případě uložena v tzv. špagetovém datovém modelu. Hlavním specifikem a úskalím tohoto modelu je úplná absence explicitně definovaných topologických vztahů – neexistují zde žádné informace o sousednosti jednotlivých polygonů či sdílení hran. Z hlediska definice problému to znamená, že polohu bodu nelze odvodit z okolí, ale je nutné ji analyzovat vůči každému polygonu v dané množině zcela nezávisle.

Popisy algoritmů formálním jazykem

Ray crossing algorithm

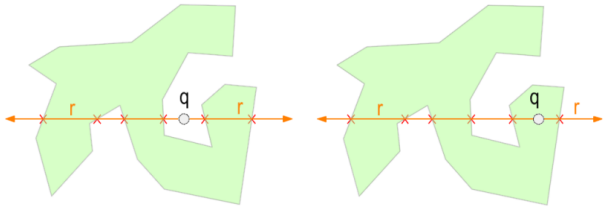
Metoda Ray Crossing slouží k určení, zda se bod q nachází uvnitř daného polygonu P. Princip spočívá ve vedení polopřímky r (paprsku) z bodu q libovolným směrem. Klíčovým ukazatelem je pak počet průsečíků k tohoto paprsku s hranicemi oblasti P. Obecně platí, že pokud je počet průsečíků lichý (v jednoduchém případě $k = 1$), bod q leží uvnitř polygonu ($q \in P$). Pokud je

počet průsečíků sudý nebo nulový ($k = 0$ nebo 2), bod leží vně polygonu ($q \notin P$).

Algoritmus je invariantní vůči volbě směru paprsku r, což znamená, že výsledek by měl být vždy stejný bez ohledu na to, kam paprsek míří. V praxi se však nejčastěji volí horizontální směr rovnoběžný s osou x (tedy přímka o rovnici $y = y_q$).

Při implementaci je však nutné dávat pozor na tzv. singulární případy, které mohou výpočet zkreslit:

- Paprsek r prochází přímo vrcholem polygonu (může dojít k započítání dvou segmentů najednou).
- Paprsek r je kolineární (rovnoběžný a totožný) s některou ze stran polygonu, což vede k nesprávnému určení počtu průsečíků.



Obrázek 1: Schéma algoritmu Ray crossing podle umístení bodu q

Aby se předešlo chybám při výpočtu v místech singularit, používá se upravená varianta algoritmu, která přesně definuje pravidla pro započítávání průsečíků. Tato modifikace využívá rozdělení prostoru hranou polygonu na horní a dolní polorovinu vzhledem k paprsku.

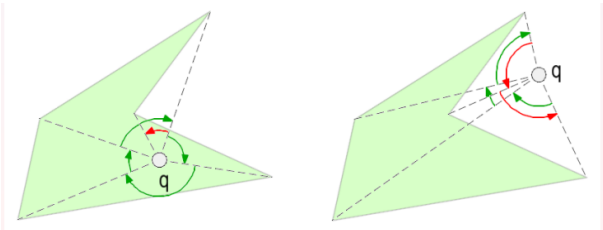
Pravidla pro inkrementaci počtu průsečíků $k(q, P)$ jsou:

- Průsečík s hranou je započítán pouze tehdy, pokud jeden koncový bod hrany leží nad paprskem ($y > y_q$) a druhý koncový bod leží na paprsku nebo pod ním ($y \leq y_q$). Tím se zajistí, že paprsek procházející vrcholem je započítán korektně právě jednou.
- Úseky hran, které leží přímo na paprsku (jsou s ním kolineární), se do celkového počtu průsečíků nezapočítávají.

Winding number algorithm

Metoda Winding Number (algoritmus úhlu oběhu) využívá k určení polohy bodu odlišný geometrický princip než vrhání paprsku. Základní myšlenka si představuje pozorovatele, který stojí

v testovaném bodě q a postupně se dívá na všechny vrcholy p_i polygonu P. Pokud se bod q nachází uvnitř polygonu ($q \in P$), pozorovatel se při sledování celého obvodu musí otočit kolem dokola, tedy o plný úhel 2π (případně o jeho násobek). Pokud bod leží vně polygonu ($q \notin P$), pohled pozorovatele se "vrátí zpět"a celkový součet úhlů se vyruší (bude menší než 2π).



Obrázek 2: Schéma algoritmu Winding number podle umístení bodu q

Výpočet spočívá v postupné sumaci orientovaných úhlů ω_i mezi body (p_i, q, p_{i+1}) přes všechny vrcholy polygonu. Pro správný výsledek je klíčové určit znaménko úhlu, které závisí na směru pohybu. K tomu se využívá orientace bodu q vůči přímce procházející aktuálním segmentem polygonu. Polohu lze analyticky určit pomocí determinantu t.

Na základě hodnoty determinantu se rovina dělí na levou a pravou polorovinu. Pokud $t > 0$, bod q leží v levé polorovině (σ_l) a úhel ω se k celkové sumě přičítá (kladná orientace proti směru hodinových ručiček – CCW). Pokud $t < 0$, bod leží v pravé polorovině (σ_r) a úhel se odečítá (záporná orientace po směru hodinových ručiček – CW).Celková hodnota oběhů Ω (Winding Number) se získá sečtením těchto úhlů. V praxi se při implementaci algoritmu počítá s nepřesnostmi reprezentace reálných čísel, a proto se netestuje absolutní rovnost, ale zavádí se malá tolerance ϵ. Podmínka pro bod uvnitř polygonu pak vypadá následovně:

$$||\Omega| - 2\pi| < \epsilon$$

V porovnání s metodou Ray Crossing má tento algoritmus tu výhodu, že přirozeně a lépe ošetřuje singulární případy (není nutné řešit složité výjimky při protnutí vrcholu). Problematická zůstává pouze situace, kdy testovaný bod přímo splývá s některým z vrcholů ($q \equiv p_i$). Hlavní nevýhodou je však vyšší výpočetní náročnost, protože algoritmus vyžaduje počítání úhlů a determinantů, což jej dělá pomalejším. Celková časová složitost je ale stále lineární, tedy $\mathcal{O}(N)$.

Problematické situace a jejich ošetření v kódu

Bod totožný s vrcholem polygonu

Z hlediska geometrie představuje vrchol polygonu 0dimenzionální simplex. Situace, kdy testovaný bod splývá s některým vrcholem, je z problematická především proto, že takový bod může být současně součástí více sousedících polygonů. V klasické variantě algoritmu Ray Crossing by navíc průchod paprsku vrcholem mohl vést k nekorektnímu započtení průsečíku. U algoritmu Winding Number by zase mohlo dojít k degeneraci některého z uvažovaných vektorů, a tím i k numerické nestabilitě při výpočtu úhlu.

Z tohoto důvodu je v implementaci detekce vrcholu provedena ještě před samotným hlavním testem polohy bodu. Pro každý vrchol polygonu je vypočtena eukleidovská vzdálenost mezi testovaným bodem q a daným vrcholem. Pokud je tato vzdálenost menší nebo rovna zvolené toleranci, je bod klasifikován jako bod ležící ve vrcholu. Implementace vrací speciální návratovou hodnotu -2, která umožňuje tento stav dále odlišit od ostatních variant, kde bod může vůči polygonu ležet (univtř, vně, na hraněú. Toto řešení je použito jak u algoritmu Ray Crossing, tak u algoritmu Winding Number. Tímto oddělujeme singularitu od samotného rozhodovacího mechanismu algoritmů.

```
for pol in poly_rings:
    n = len(pol)
    for i in range(n):
        #Check vertex hit
        dist_v = sqrt((q.x() - pol[i].x())**2 + (q.y() - pol[i].y())**2)
        if dist_v <= self.tolerance:
            return -2
```

Obrázek 3: Řešení situace, kdy je bod totožný s vrcholem polygonu

V případě, že je bod vyhodnocen jako totožný s vrcholem, aplikace zvýrazní všechny polygony, které tento vrchol sdílejí. Tato logika je implementována v hlavní třídě aplikace při vyhodnocení návratových stavů algoritmu a předání seznamu polygonů vykreslovací třídě. Uživatel tak nezíská pouze textovou informaci, ale i grafickou interpretaci situace.

Bod ležící na hraně polygonu

Hrana polygonu představuje 1-dimenzionální simplex, i tento případ je nutné řešit samostatně. Pokud by taková situace nebyla explicitně detekována, mohl by být bod v závislosti na numerické přesnosti výpočtu a konfiguraci segmentů klasifikován nekonzistentně. V implementaci je tento problém řešen výpočtem minimální vzdálenosti bodu od každého segmentu polygonu. Nejprve je sestrojen směrový vektor hrany a vektor od počátečního bodu hrany k testovanému bodu. Následně je určena projekce bodu na přímku nesoucí hranu. Hodnota parametru projekce je omezena do intervalu $\langle 0, 1 \rangle$, čímž se zajistí, že se hledá nejbližší bod skutečně na úsečce. Poté je vypočtena vzdálenost mezi testovaným bodem a tímto nejbližším bodem na hraně. Pokud je tato vzdálenost menší nebo rovna toleranci, je bod klasifikován jako ležící na hraně a algoritmus vrací hodnotu -1. Tento způsob je opět použit v obou implementovaných algoritmech.

Namísto testování přesné kolinearity a současně ležení bodu v mezích segmentu, což bývá citlivé na zaokrouhlovací chyby, je použito metrické kritérium založené na vzdálenosti. To lépe odpovídá i praktickému použití v aplikaci, kde je poloha bodu zadávána kliknutím myši a nelze předpokládat přesné umístění. Stejně jako v případě vrcholů je i zde stav dále aplikován na úrovni MainForm. Pokud je detekováno, že bod leží na hraně, aplikace seskupí všechny polygony, u nichž byl tento stav zjištěn, a graficky je zvýrazní.

```
if len(found_polygons_in) > 1:
    mb.setText(f"Point is INSIDE {len(found_polygons_in)} overlapping polygons")
    self.Canvas.setHighlightedPolygons(found_polygons_in)
elif len(found_polygons_in) == 1:
    mb.setText("Point is INSIDE the polygon")
    self.Canvas.setHighlightedPolygons(found_polygons_in)
elif len(found_polygons_vert) > 0:
    mb.setText(f"Point is exactly on a VERTEX shared by {len(found_polygons_vert)} polygon(s)")
    self.Canvas.setHighlightedPolygons(found_polygons_vert)
elif len(found_polygons_edge) > 0:
    mb.setText(f"Point is on a BOUNDARY shared by {len(found_polygons_edge)} polygon(s)")
    self.Canvas.setHighlightedPolygons(found_polygons_edge)
else:
    mb.setText("Point is OUTSIDE all polygons")
    self.Canvas.setHighlightedPolygons([])
```

Obrázek 4: Ukázka kódu pro identifikaci polohy bodu a polygonu

Polygony s dírami

Z hlediska datové reprezentace i interpretace pro nás byly polygony s dírami nejkomplexnějším případem celé úlohy. Takový polygon totiž není tvořen pouze jednou uzavřenou hranicí, ale skládá se z více kružnic (rings), přičemž jedna z nich reprezentuje vnější obrys a ostatní vymezují oblasti, které do polygonu nenáleží. Nestačí posuzovat pouze to, zda bod leží uvnitř vnější

hranice polygonu, ale je nutné současně ověřit, zda se nenachází uvnitř některé z jeho vnitřních částí. Tento problém jsme řešili již na úrovni načítání vstupních dat. Při zpracování formátu Shapefile jsme jednotlivé části polygonu (tj. jeho prstence) načítali samostatně a ukládali je jako seznam více kružnic. Díky tomu jsme zachovali kompletní topologickou strukturu polygonu a mohli s ní dále pracovat v celém průběhu aplikace.

Takto získanou strukturu jsme následně předávali jak do výpočetních algoritmů, tak do vykreslovací části programu. V algoritmech jsme jednotlivé prstence polygonu procházeli postupně. Zásadní roli hrála také vizualizace. Při vykreslování jsme využili objekt QPainterPath s pravidlem vyplňování Odd-Even Fill, které odpovídá principu sudého a lichého počtu průchodů hranicemi. Díky tomu zůstávají vnitřní kružnice (díry) nevyplněné a polygon je zobrazen korektně.

Stejný princip jsme aplikovali i při zvýraznění výsledného polygonu, čímž jsme zajistili konzistentní chování jak v analytické, tak ve vizuální části aplikace.

```
# We iterate through the individual parts of the polygon (islands, holes, etc.).
record_polygons = []
for p in range(num_parts):
    part_points = []
    start_pt = parts_indices[p]
    end_pt = parts_indices[p+1]

    # Loading points for a specific part (properly indented)
    for i in range(start_pt, end_pt):
        start = points_start_index + (i * 16)
        x, y = struct.unpack("<dd", record_content[start:start+16])
        part_points.append([x, y])

    #Add the loaded points to the list of parts of this polygo
    if len(part_points) > 0:
        record_polygons.append(np.array(part_points))

if record_polygons:
    polygons.append(record_polygons)
```

Obrázek 5: Část kódu pro rozklad polygonu na jednotlivé části a extrakci souřadnic

```
for poly_rings in self.__pols:
    path = QPainterPath()
    path.setFillRule(Qt.FillRule.OddEvenFill)
    for ring in poly_rings:
        path.addPolygon(ring)
    qp.drawPath(path)
```

Obrázek 6: Vykreslovací smyčka pro geometrické tvary v grafickém okně aplikace

Vstupní data

Pro běh a testování implementovaných algoritmů (Ray Crossing a Winding Number) byla zvolena reálná geografická data, konkrétně se jedná o kraje ČR. Tato data obsahují detailní geometrii hranic, což z nich dělá ideální a dostatečně komplexní vstup pro testování problému polohy bodu v nekonvexních oblastech. Hlavní město Praha zde tvoří zastoupení polygonu s dírou. Vstupní data jsou poskytována ve standardním vektorovém formátu ESRI Shapefile (.shp).

Zásadním bodem této části implementace bylo samotné zpracování vstupních souborů. Namísto využití vysokoúrovňových externích knihoven, které by prostorová data načetly automaticky, byl proces čtení formátu Shapefile přímo naprogramován. Tento postup vyžadoval vytvoření vlastního parseru pro čtení binárních dat. Implementovaný algoritmus nejprve načítá hlavní hlavičku souboru pro zjištění metadat a globálního obalového obdélníku (bounding box). Následně sekvenčně prochází jednotlivé záznamy (kraje), u kterých analyzuje jejich typ a iterativně načítá samotné souřadnice lomových bodů (vrcholů), jež tvoří hranice polygonů.

Takto extrahované souřadnice jsou v reálném čase transformovány a ukládány do vnitřních datových struktur aplikace. Tím je zajištěno, že jsou prostorová data z formátu Shapefile úspěšně převedena do podoby, nad kterou mohou následně efektivně operovat naše lokalizační algoritmy.

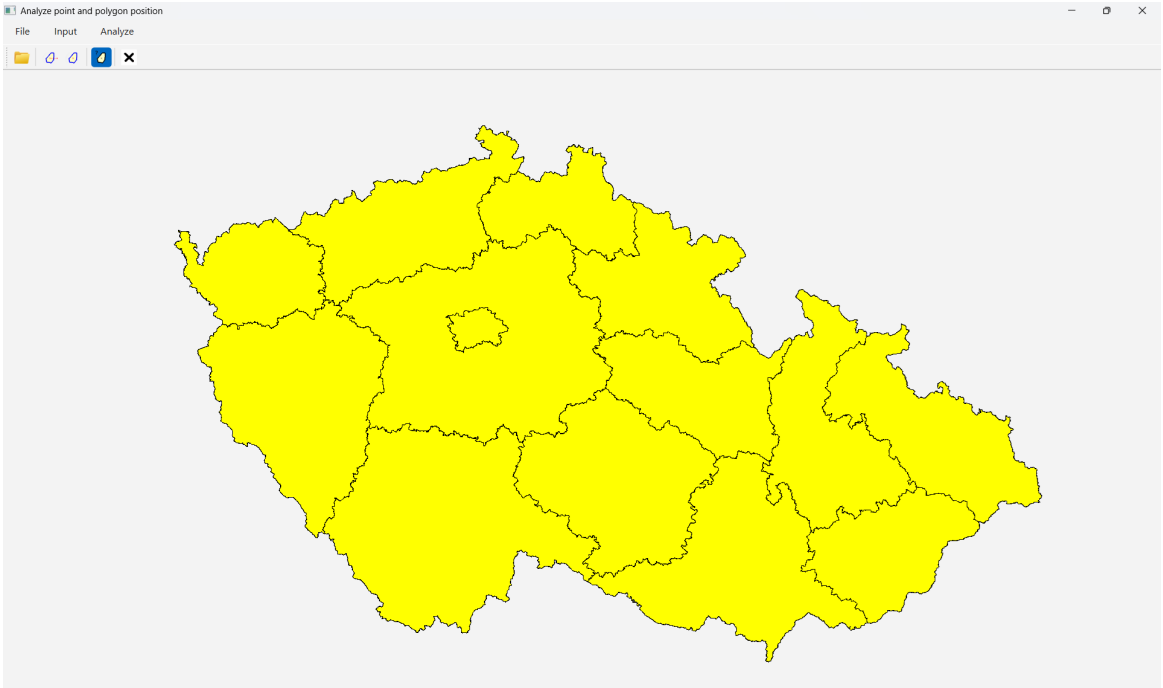
Výstupní data

Výstupem implementované aplikace není datový soubor v klasickém smyslu, ale především vizuální a textová interpretace výsledku analýzy. Cílem je jednoznačně určit polohu zadaného bodu vzhledem k polygonové mapě a tuto informaci přehledně prezentovat. Základním výstupem je klasifikace testovaného bodu q vůči jednotlivým polygonům. V závislosti na výsledku algoritmu, jak již bylo, řečeno, může být bod vyhodnocen jako:

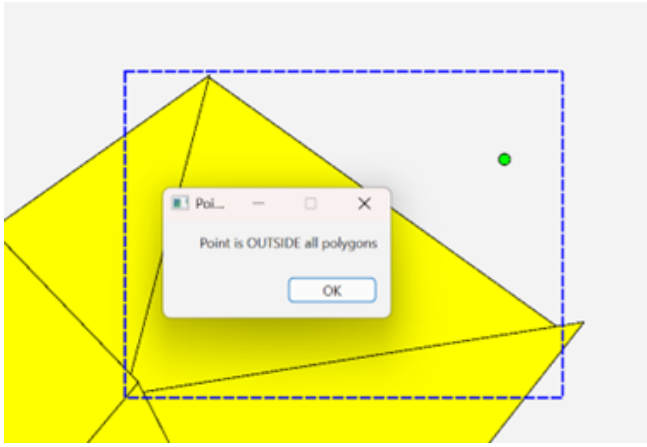
- ležící uvnitř polygonu
- ležící vně polygonů
- ležící na hraně polygonu
- totožný s vrcholem polygonu

Tato informace je sdělena prostřednictvím dialogového okna, které obsahuje slovní popis výsledku. V případě, že bod leží na hraně nebo ve vrcholu sdíleném více polygony, aplikace tuto skutečnost explicitně uvádí a informuje o počtu dotčených objektů. Vedle textového výstupu hraje roli také grafická reprezentace výsledku. Polygon (nebo více polygonů), který odpovídá výsledku analýzy, je zvýrazněn přímo v mapovém okně. Zvýraznění je realizováno změnou barvy výplně, čímž je dosaženo okamžité vizuální identifikace příslušného objektu. Součástí výstupu je rovněž vizualizace optimalizačního kroku pomocí min-max boxů. Pro všechny kandidátní polygony, které prošly rychlým testem obalového obdélníku, jsou tyto boxy vykresleny přerušovanou čarou. Uživatel tak získává i přehled o průběhu výpočtu a redukci vyhledávacího prostoru.Z hlediska formátu lze tedy výstup charakterizovat jako interaktivní vizualizaci kombinující grafickou a textovou složku.

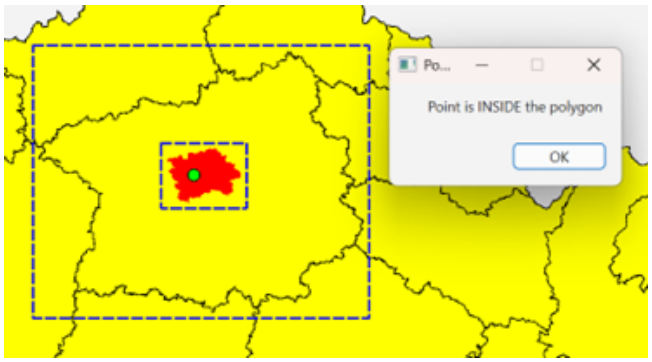
Vizualizace vytvořené aplikace



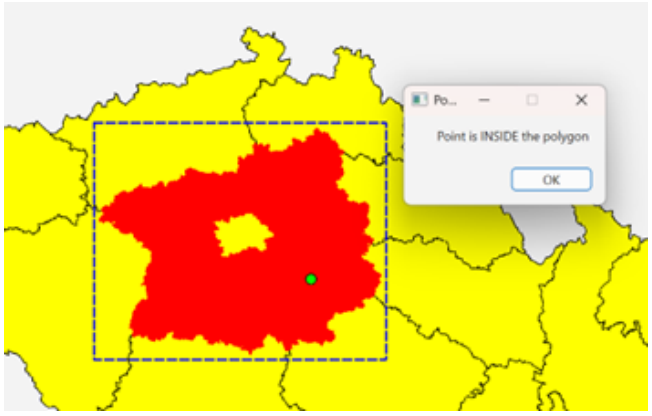
Obrázek 7: Načtení vstupních dat (krajů ČR)



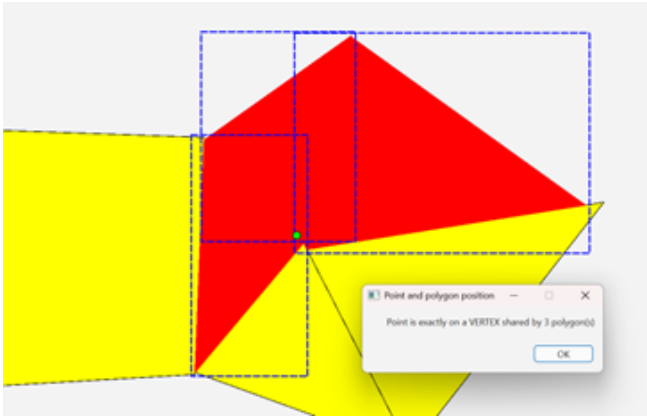
Obrázek 8: Výstup pro situaci, kdy je bod mimo polygon



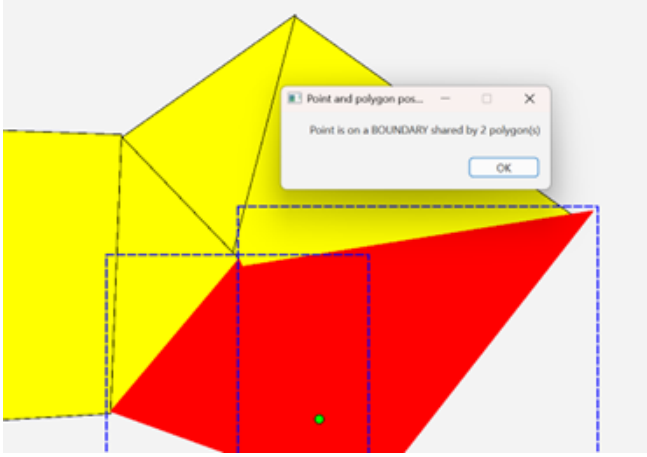
Obrázek 9: Bod je uvnitř polygonu



Obrázek 10: Situace pro polygon s dírou



Obrázek 11: Bod je na vrcholu, jež sdílí více polygonů



Obrázek 12: Bod je na hraně dvou polygonů

Dokumentace tříd a použitých metod

Implementace aplikace je rozdělena do tří hlavních skriptů: algorithms.py, draw.py a MainForm.py. Rozdělení odpovídá logickému oddělení výpočetní části, prezentační vrstvy a řídicí logiky aplikace.

Skript algorithms.py

Skript algorithms.py obsahuje třídu Algorithms, která implementuje výpočetní operace související s určením polohy bodu vůči polygonu. Třída obsahuje hlavní položku tolerance která slouží k ošetření numerických nepřesností při výpočtech.

createMinMaxBox(poly_rings)

Metoda slouží k výpočtu obalového obdélníku (bounding boxu) pro daný polygon. Prochází všechny vrcholy všech prstenců polygonu a určuje minimální a maximální souřadnice v osách x a y. Tento box je využíván jako optimalizační krok pro rychlé vyloučení polygonů, které nemohou obsahovat testovaný bod.

getPointPolygonPositionRC(q, poly_rings)

Implementace algoritmu Ray Crossing. Princip spočívá v počítání průsečíků paprsku vedeného z testovaného bodu s hranami polygonu. Nejprve se kontroluje, zda bod neleží ve vrcholu polygonu. Následně se testuje, zda bod neleží na některé hraně polygonu. Pokud bod neleží ani na hraně, ani ve vrcholu, provede se samotný Ray Crossing test. Výstupem metody je: 1 – bod leží uvnitř polygonu, 0 – bod leží vně polygonu, -1 – bod leží na hraně polygonu, -2 – bod splývá s vrcholem polygonu

getPointPolygonPositionWinding(q, poly_rings)

Implementace algoritmu Winding Number. Tento algoritmus vyhodnocuje celkový úhel, který polygon „opíše“ kolem testovaného bodu. Nejprve se zase ošetřují singulární případy (vrchol, hrana). Následně se pro každou dvojici sousedních vrcholů vypočítá orientovaný úhel. Úhly se sčítají s ohledem na orientaci a pokud absolutní hodnota výsledného úhlu odpovídá hodnotě 2, bod se nachází uvnitř polygonu.

Skript draw.py

Skript draw.py obsahuje třídu Draw, která zajišťuje vykreslování polygonů, testovaného bodu a výsledků analýzy. __pols je seznam polygonů, kde každý polygon může obsahovat více prstenců (vnější hranice a díry), __q je testovaný bod. Highlighted_polygons je seznam polygonů, které

mají být graficky zvýrazněny a minmax_boxes je seznam obalových obdélníků pro vizualizaci optimalizace.

mousePressEvent(e) zpracovává kliknutí myši.

paintEvent(e)

Hlavní vykreslovací metoda. Postupně vykresluje všechny polygony (žlutá výplň), zvýrazněné polygony (červená), min-max boxy (modrá přerušovaná čára) a testovaný bod (zelený). Pro vykreslení polygonů s dírami je použit QPainterPath s pravidlem Odd-Even Fill.

drawPolygons(polygons)

Převádí vstupní data (NumPy pole) na objekty QPolygonF.

setHighlightedPolygons(pols) a *setMinMaxBoxes(boxes)*

Slouží k aktualizaci grafické reprezentace výsledků analýzy.

clearData()

Vymaže všechna data a resetuje stav aplikace.

Skript MainForm.py

Skript MainForm.py obsahuje třídu Ui_MainForm. Třída Ui_MainForm představuje řídicí vrstvu aplikace. Zajišťuje komunikaci mezi uživatelem, výpočetní částí a vykreslovací komponentou. Obsahuje logiku načítání dat, spuštění analýzy i prezentaci výsledků. Datové položky jsou Canvas, která slouží k instanci třídy Draw, (sloužící jako hlavní vykreslovací plocha) a GUI komponenty – menu, toolbar a akce pro ovládání aplikace.

getPolygonsFromSHP(file_path)

Implementuje čtení binárního formátu Shapefile bez použití externích knihoven. Načítá jednot-

livé části polygonu (rings) a ukládá je do vnitřní struktury.

normalizeData()

Transformuje souřadnice polygonů tak, aby se vešly do zobrazovacího okna. Provádí škálování a centrování dat.

analyzePointAndPositionClick(method)

Hlavní metoda aplikace pro analýzu polohy bodu. Nejdříve načte bod a seznam polygonů. Pro každý polygon vytvoří min-max box, provede rychlý test, a aplikuje zvolený algoritmus (Ray Crossing nebo Winding Number). Dále vyhodnotí výsledky a také zajistí grafické zvýraznění a zobrazí výsledků v dialogovém okně.

analyzeRC() a *analyzeWN()*

Jedná se o obslužné metody pro spuštění konkrétního algoritmu.

changeStatusClick() a *clearClick()*

Zajišťují interakci s uživatelem – přepínání režimu a reset aplikace.

Závěr

V rámci této práce jsme implementovaly řešení úlohy lokalizace bodu v polygonové mapě (Point-in-Polygon Problem) s využitím algoritmů Ray Crossing a Winding Number. Součástí řešení bylo také ošetření singulárních případů, práce s polygonovými daty ve formátu Shapefile a implementace jednoduchého grafického rozhraní v prostředí PyQt.Navržené řešení umožňuje korektně určovat polohu bodu i pro komplexnější geometrie, včetně polygonů s dírami, a poskytuje okamžitou vizuální zpětnou vazbu. Implementace min-max boxu zároveň přispívá ke zrychlení výpočtu tím, že omezuje množinu analyzovaných polygonů. Přestože aplikace splňuje požadavky zadání, lze identifikovat několik oblastí, které by bylo možné dále rozšířit či vylepšit.

Jedním z možných vylepšení je úprava uživatelského rozhraní tak, aby při změně polohy testovaného bodu došlo automaticky k vymazání předchozího výsledku. V současné implementaci

zůstává zvýraznění předchozího polygonu zachováno, což může být pro uživatele matoucí. Dalším krokem by mohlo být plně automatické spuštění analýzy bez nutnosti manuálního výběru algoritmu. Analýza by se tak prováděla ihned po změně polohy bodu, což by vedlo k plynulejšímu a intuitivnějšímu ovládání aplikace.

Z algoritmického hlediska by bylo přínosné provést detailnější porovnání implementovaných metod Ray Crossing a Winding Number z hlediska výpočetní náročnosti, přesnosti a stability výsledků. Zajímavým rozšířením by bylo také grafické znázornění průběhu algoritmů, například vykreslení paprsku u metody Ray Crossing nebo vizualizace jednotlivých úhlů u Winding Number, což by přispělo k lepšímu pochopení jejich principu.

Z hlediska vstupních dat by bylo přínosné rozšířit podporu o další formáty (např. GeoJSON) a umožnit současné načtení více vrstev nad sebou. Tím by bylo možné analyzovat složitější prostorové vztahy mezi objekty z různých datových sad.Další potenciální rozšíření spočívá v implementaci pokročilejších prostorových struktur (např. R-tree), které by umožnily efektivnější vyhledávání kandidátních polygonů a tím i zrychlení výpočtu u rozsáhlých datových souborů.

Seznam literatury

BAYER, T. (2026): Algoritmy počítačové kartografie, Point location problem, 2. přednáška, Přírodovědecká fakulta, Karlova Univerzita.

RIVERBANK COMPUTING (2026): PyQt6 Documentation, dostupné z: https://www.riverbankcomputing.com/static/Docs/PyQt6/index.html